

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – 5

Roll No.: T050	Name: AASHU YADAV
Class: TYIT	Batch: 1
Date of Assignment: 28/07/2026	Date/Time of Submission: 28/07/2026

Part B - Program

a) Create a web application to demonstrate the use of different types of Cookies.

Algorithm:

Step1:-Start.

Step2:-Enter user data.

Step3:-Create and store cookies.

Step4:-Read and display cookies.

Step5:-Update or delete cookies if required.

Code:-

Webform1.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
namespace practical_5
{
    public partial class WebForm1 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if(IsPostBack)
            {
                int ViewStateVal =
Convert.ToInt32(ViewState["count"]) + 1;
                Label2.Text = "View State =
"+ViewStateVal.ToString();
            }
        }
    }
}
```

Webform2.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
```

```
else
{
    ViewState["count"] = 1;
}
protected void Button1_Click(object sender,
EventArgs e)
{
    Label1.Text=ViewState["count"].ToString();
}
protected void Button2_Click(object sender,
EventArgs e)
{
    HttpCookie h = new HttpCookie("name");
h.Value = TextBox1.Text;
Response.Cookies.Add(h);
Response.Redirect("Webform2.aspx");
}
}
}

namespace practical_5
{
    public partial class WebForm2 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
        }
    }
}
```

Aashu Yadav
T050

```
        if (Request.Cookies["name"]!=null)
        {
            Response.Write("Welcome "+Request.Cookies["name"].Value);
        }
    }
```

Webform1.aspx

body

Label

Label

View state

cookies

Webform2.aspx

```
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
  <title></title>
</head>
<body>
  <form id="form1" runat="server">
    <div>
      <asp:Label ID="Label1" runat="server" Text="Label"></asp:Label>
    </div>
  </form>
</body>
</html>
```

Output:-

Label	1	
Label	View State = 2	
View state	View state	Welcome Aashu Yadav Label
	Aashu Yadav	
cookies	cookies	

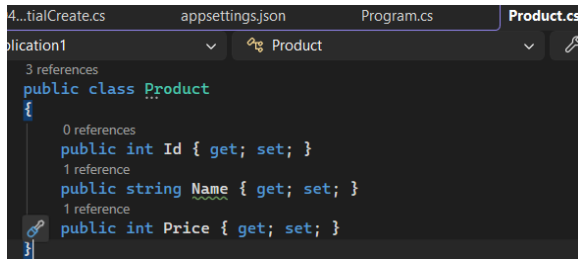
b) Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.

Algorithm:

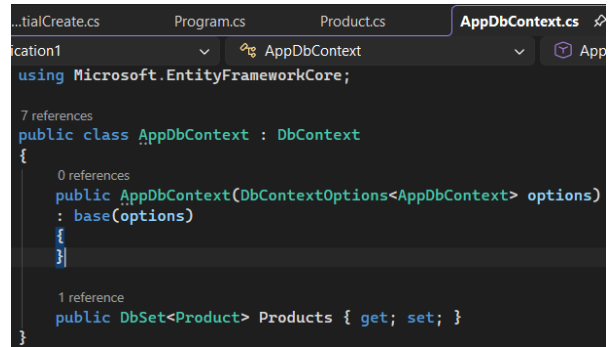
- step1:-Start.
- step2:-Create a .NET project.
- step3:-Add EF Core packages.
- step4:-Create the Product model and DbContext.
- step5:-Configure the SQL Server connection.
- step6:-Run migrations to create the table.
- step7:-Insert a product using Add() and SaveChanges().
- step8:-Verify the record.

Aashu Yadav
T050

step9:-Stop.



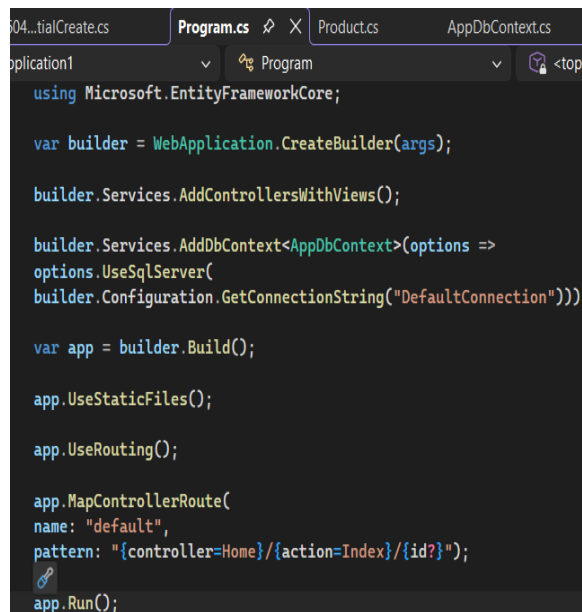
```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Price { get; set; }
}
```



```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
    : base(options)
    {
    }

    public DbSet<Product> Products { get; set; }
}
```

Code:



```
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(
builder.Configuration.GetConnectionString("DefaultConnection")))

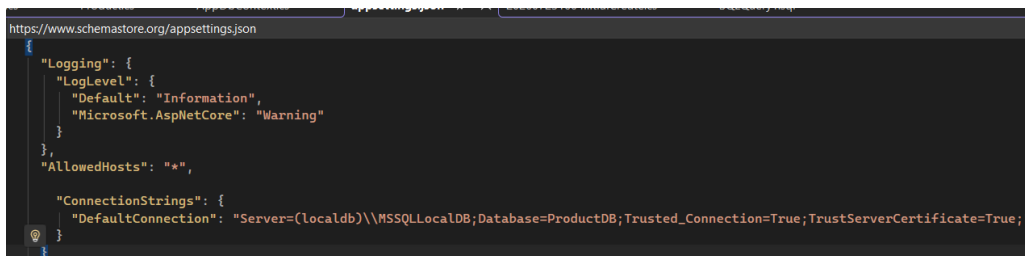
var app = builder.Build();

app.UseStaticFiles();

app.UseRouting();

app.MapControllerRoute(
name: "default",
pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```



```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=ProductDB;Trusted_Connection=True;TrustServerCertificate=True;"
  }
}
```

```
Product.cs  AppDbContext.cs  appsettings.json
Application1  HomeController

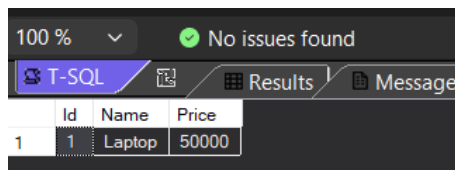
0 references
public HomeController(AppDbContext context)
{
    _context = context;
}

0 references
public IActionResult Index()
{
    Product p = new Product();
    p.Name = "Laptop";
    p.Price = 50000;

    _context.Products.Add(p);
    _context.SaveChanges();

    return Content("Product Inserted Successfully");
}
```

Output:



Id	Name	Price
1	Laptop	50000

Product Inserted Successfully

c) Develop a web application to perform CRUD Operations (Create, Read, Update, Delete) on Employee table using EF Core.

Algorithm:

Steps :

1. Define Model: Create Employee.cs with properties (Id, Name, Department, Salary).
2. Setup DbContext: Create AppDbContext.cs with DbSet<Employee> Employees and register it inside Program.cs.
3. Apply Migrations: Run Add-Migration AddEmployeeTable followed by Update-Database in Package Manager Console.
4. Build Controller Actions: In EmployeesController.cs, implement methods for Create, Read, Update, and Delete operations.
5. Create Views: Generate corresponding Razor views under Views/Employees/ (Index, Create, Edit, Delete).

Employee.cs

```
namespace EmployeeCRUD.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public required string Name { get; set; }
        public required string Department { get; set; }
    }

    public double Salary { get; set; }
}
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
```

```
using EmployeeCRUD.Data;
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<AppDbContext>
(options =>
options.UseSqlServer(builder.Configuration.Get
ConnectionString("MyConnection")));
```

appsettings.json:

```
"AllowedHosts": "*",
"ConnectionStrings": {
    "MyConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=Em
ployeeDB;Trusted_Connection=True;TrustServer
Certificate=True",
```

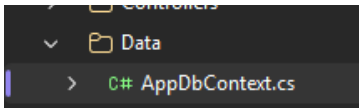
Aashu Yadav
T050

```
"AppDbContext":  
"Server=(localdb)\\mssqllocaldb;Database=App  
DbContext;Trusted_Connection=True;MultipleA  
ctiveResultSets=true"  
}
```

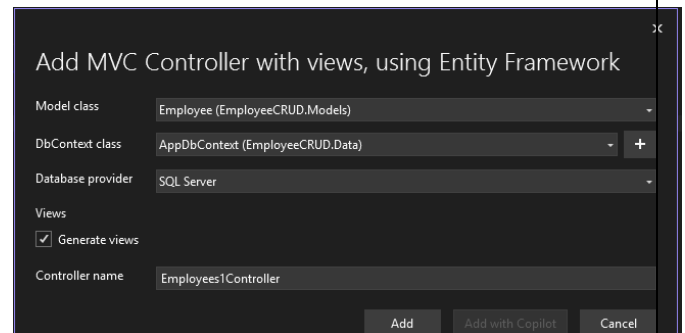
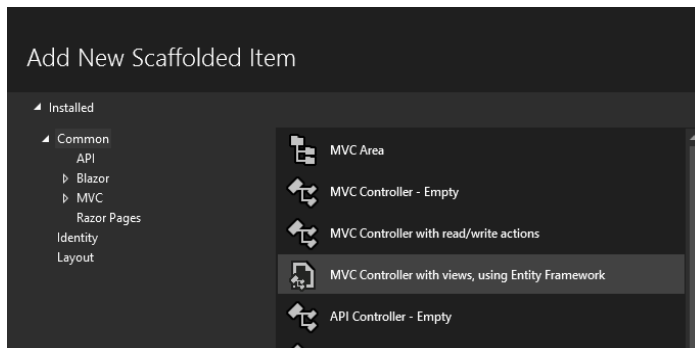
AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;  
using EmployeeCRUD.Models;  
namespace EmployeeCRUD.Data  
{
```

```
public class AppDbContext:DbContext  
{  
    public  
    AppDbContext(DbContextOptions<AppDbConte  
xt> options) : base(options) { }  
    public DbSet <Employee> Employees { get;  
set; }  
}
```



Controllers->Add-> New Scaffolded Item



Output:-

EmployeeCRUD Home Privacy

Welcome

Learn about [building Web apps with ASP.NET Core](#).



EmployeeCRUD Home Privacy

Create

Employee

Name

Department

Salary

Create

[Back to List](#)

Index

[Create New](#)

Name	Department	Salary	
Akansha	IT	90000	Edit Details Delete

d) Authentication and Authorization in ASP.NET Core.

Algorithm:

step1:-Create an ASP.NET Core MVC project.

step2:-Enable Cookie Authentication in Program.cs.

step3:-Create Login action in HomeController.

step4:-Verify username and password.

step5:-Redirect to Secure page after successful login.

step6:-Use [Authorize] to protect the Secure page.

step7:-Logout and redirect to Login.

Code:-

Program.cs

```
using Microsoft.AspNetCore.Authentication.Cookies;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
builder.Services.AddAuthentication(CookieAuthentication
Defaults.AuthenticationScheme)
.AddCookie(options =>
{
options.LoginPath = "/Home/Index";
});
```

HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using System.Security.Claims;
namespace AuthDemo.Controllers
```

```
builder.Services.AddAuthorization();
var app = builder.Build();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllerRoute(
name: "default",
pattern:
"/{controller=Home}/{action=Index}/{id?}");
app.Run()
{
public class HomeController : Controller
{
public IActionResult Index()
{
return View();
}
```

[HttpPost]

```
public async Task<IActionResult> Index(string
username, string password)
{
    if (username == "admin" & &
password == "123")
    {
        var claims = new List<Claim>
        {
            new Claim(ClaimTypes.Name, username)
        };
    }
```

[Authorize]

```
public IActionResult Secure()
{
    return Content("Welcome! Login Successful.");
}
public async Task<IActionResult> Logout()
```

Views/Home/Index.cshtml

```
<h2>Login</h2>
<form method="post">
    Username :
    <input type="text"
name="username" /><br />
    Password :
```

Output:-

Login

Username:

Password:

Login

```
var identity = new ClaimsIdentity(claims,
CookieAuthenticationDefaults.AuthenticationScheme);
await HttpContext.SignInAsync(
CookieAuthenticationDefaults.AuthenticationScheme,
new ClaimsPrincipal(identity));
return RedirectToAction("Secure");
}
ViewBag.Message = "Invalid Username or
Password";
return View();
}
{
    await HttpContext.SignOutAsync(
CookieAuthenticationDefaults.AuthenticationScheme);
    return RedirectToAction("Index");
}
}
<input type="password"
name="password" /><br />
<input type="submit"
value="Login" />
</form>
<p style="color:red">
@ViewBag.Message
</p>
```

Welcome! Login Successful.